

資料結構

Data Structure

Lab 12

姓名：夏宇柔
學號：113840005

Lab12-Ex1.

Sort the products according to the rules specified and output the products from the most prominent position to the least prominent position. You may choose and implement any algorithm to solve this problem.

Code

```
#include <iostream>
#include <fstream>
#include <vector>
#include <string>
using namespace std;

// 商品結構
struct Product {
    string name;
    int expiry;      // 有效期限 (天)
    int popularity; // 熱銷程度 (1~10)

    // 自訂比較規則 (小頂堆)
    bool operator<(const Product& other) const {
        if (expiry != other.expiry)
            return expiry < other.expiry; // 有效期限越小越優先
        return popularity > other.popularity; // 熱銷程度越高越優先
    }
};

vector<string> insertionSort(vector<Product>& products) { // insertion sort
    algorithm: sorts in-place
    int i = 0, j = 0, k = 0, n=products.size(); // initializes i,j and k
    counters, retrieves products.size() once as n
    for (i = 1; i < n; i++) { // start looping through every product,
        starting from the second
        k = i; // k is the index of the product we are currently processing
        for (j = i - 1; j >= 0 && products[k] < products[j]; j--) { // j is
            the index of the products already sorted, starting from the last one before
            current product
            swap(products[k], products[j]); // if product k has smaller
            expiry/higher popularity than product j, swap them
            k = j; // update product k's new index
        }
    }
    vector<string> output; // initialize output vector
    for (int i = 0; i < n; i++) {
        output.push_back(products[i].name); // pushes the name of each sorted
        product into the vector
    }
    return output;
}

int main() {
    ifstream infile("input3.txt");
    if (!infile) {
        cerr << "無法打開檔案 products.txt" << endl;
        return 1;
    }
}
```

```

int N;
infile >> N;
infile.ignore(); // 忽略换行符

vector<Product> products;
for (int i = 0; i < N; ++i) {
    string line;
    getline(infile, line);
    size_t lastSpace = line.rfind(' ');
    size_t secondLastSpace = line.rfind(' ', lastSpace - 1);

    string name = line.substr(0, secondLastSpace);
    int expiry = stoi(line.substr(secondLastSpace + 1, lastSpace -
secondLastSpace - 1));
    int popularity = stoi(line.substr(lastSpace + 1));

    products.push_back({ name, expiry, popularity });
}

infile.close();

vector<string> sortedNames;
sortedNames = insertionSort(products);

// 輸出
cout << "商品排序:" << endl;
for (const string& name : sortedNames) {
    cout << name << endl;
}

return 0;
}

```

Discussion Section

In the worst case scenario(the vector is in reversed order), the time complexity is $O(N^2)$, in the best case scenario(the vector is already in sorted order), the time complexity is $O(N)$. On average, it is $O(N^2)$.

I chose insertion sort because it is closer to the real-life scenario; it's the method I would have been using to sort the products.

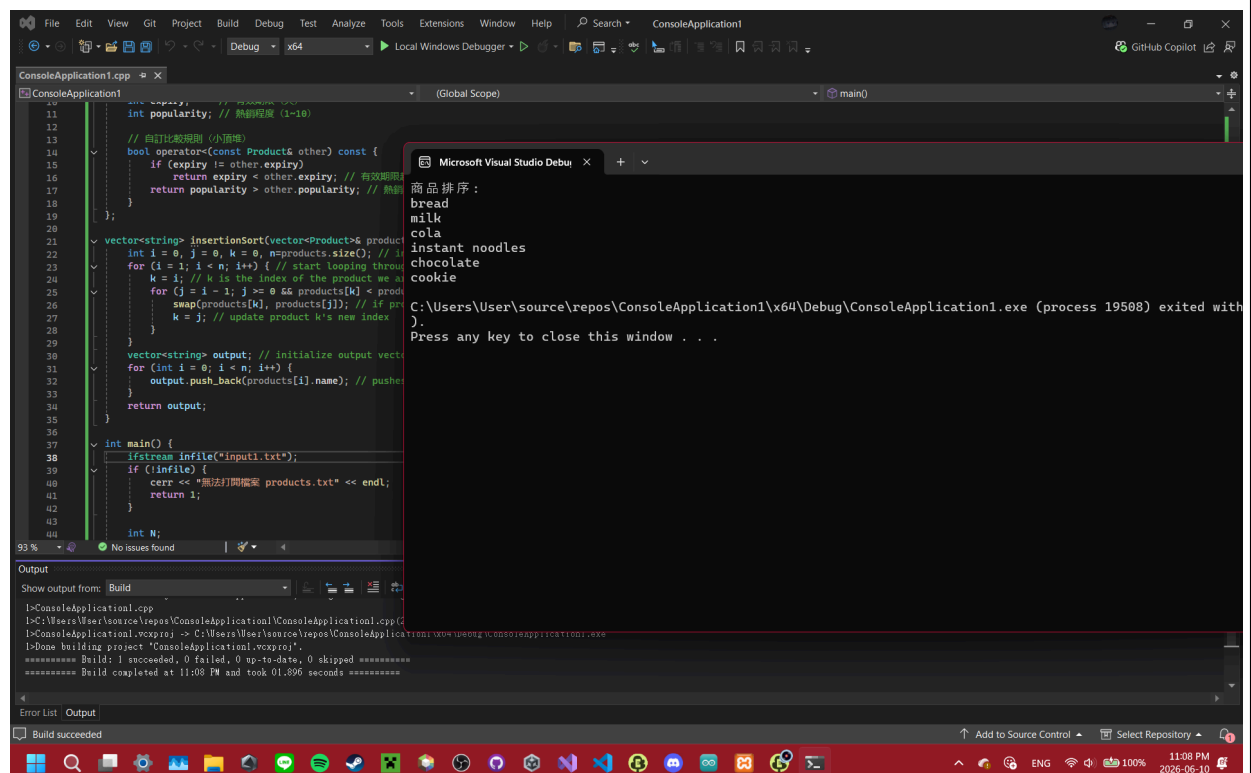
Screenshots:

input1.txt

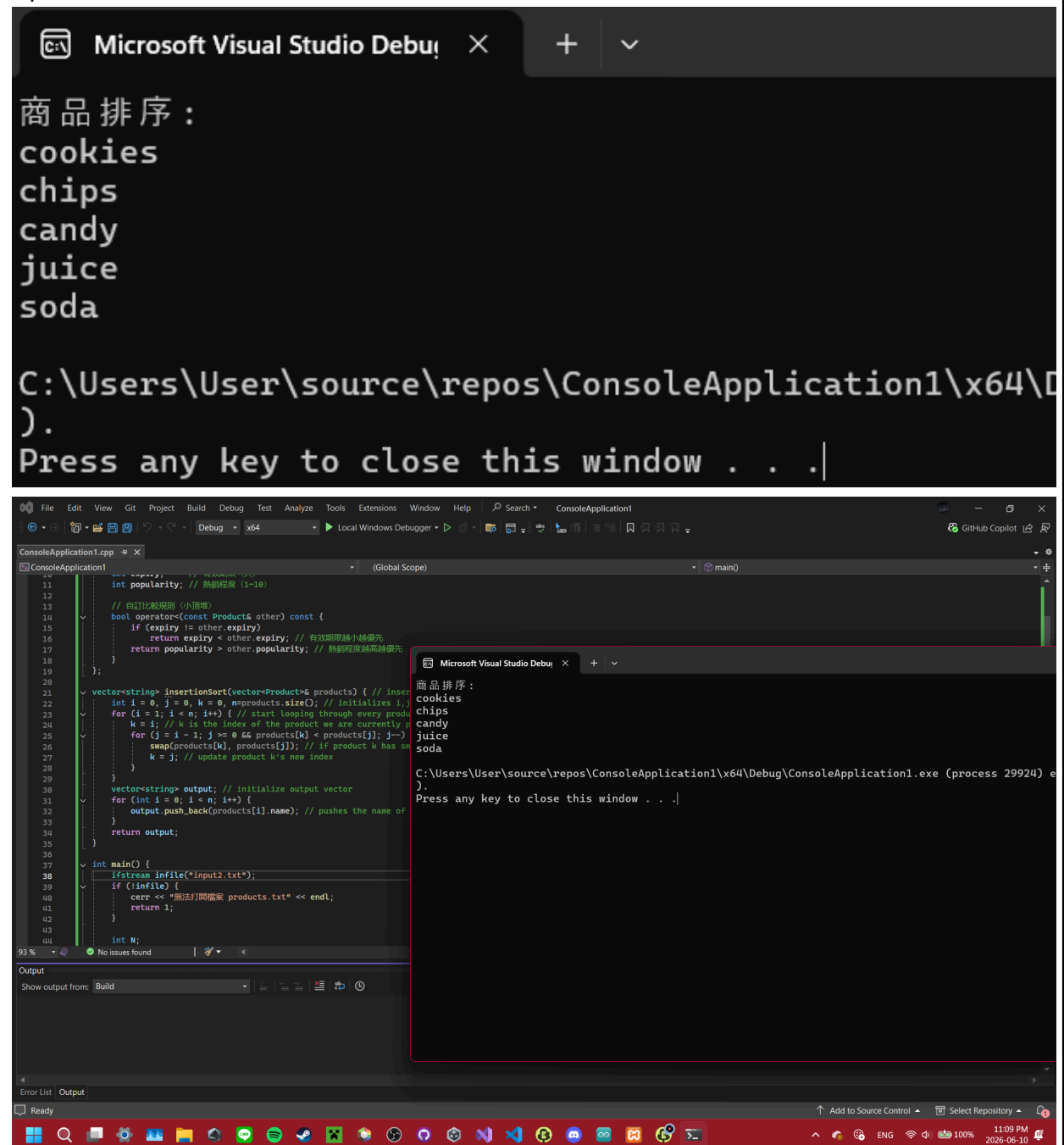
```
Microsoft Visual Studio Debug Console

商品排序：
bread
milk
cola
instant noodles
chocolate
cookie

C:\Users\User\source\repos\ConsoleApplication1\x64\Debug\ConsoleApplication1.exe (process 19508) exited with code 0.
Press any key to close this window . . .
```



input2.txt



input3.txt

